

Chapter 4 Classification

Austin R.J. Downey

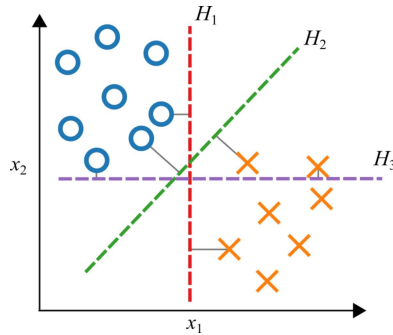
Classification

Machine learning tasks often fall into two categories:

- Regression
 - Predict continuous values
- Classification
 - Predict discrete classes

In this chapter we focus on:

- Binary classification
- Multiclass classification
- Performance metrics



Linear decision boundaries separating two classes.

Binary Classifier

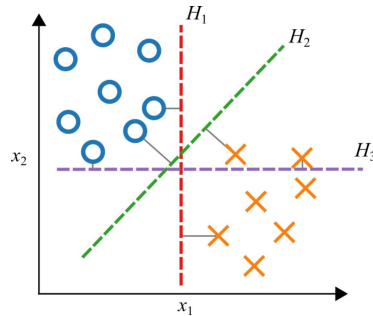
A binary classifier assigns inputs to one of two classes.

Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \theta^T \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- $\mathbf{x}$: input features
- θ : model weights
- b : bias term

Training adjusts θ and b to minimize classification error.



Multiple linear decision boundaries can correctly separate the data.

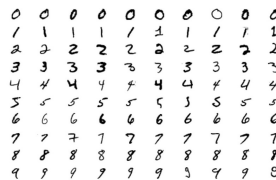
Data: MNIST Dataset

The MNIST dataset contains:

- 70,000 handwritten digits
- Classes 0 through 9
- Standard benchmark for classification

Typical split:

- 60,000 training images
- 10,000 testing images



Examples of MNIST digits.

Data: What is a Feature?

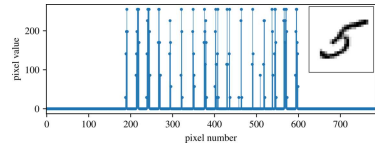
Each MNIST image is:

$$28 \times 28 = 784$$

pixels.

In machine learning:

- Each pixel is a feature
- Pixel intensity ranges from 0 to 255
- One image becomes a vector with 784 features



Each pixel becomes a feature used by the classifier.

For MNIST:

$$\mathbf{x} = [x_1, x_2, \dots, x_{784}]$$

Example 4.1: MNIST Dataset

To simplify the classification problem, we begin by building a: **5-detector**

Goal:

- Determine whether a digit is a 5
- Binary output:
 - 1 = digit is a 5
 - 0 = digit is not a 5

Why choose 5s?

- One of the most difficult digits to classify
- Frequently confused with 3s and 8s
- Good benchmark for binary classification

This example demonstrates how to use:

`sklearn.datasets.fetch_openml`

to load and visualize the MNIST dataset.

Regularized Linear Classifier

A practical way to build the MNIST “5-detector” is with a regularized linear classifier trained using Stochastic Gradient Descent.

- Efficient on large datasets
- Simple to implement
- Updates one example at a time

Main drawbacks:

- Needs careful hyperparameter tuning
- Sensitive to feature scaling

Training Objective

Given labeled samples $(\mathbf{x}^{(i)}, y^{(i)})$ with $y^{(i)} \in \{0, 1\}$, the “5-detector” learns parameters by minimizing:

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \max(0, 1 - y^{(i)} \theta^T \mathbf{x}^{(i)}) + \frac{\lambda}{2} \|\theta\|_2^2$$

- First term: hinge loss
- Second term: ℓ_2 regularization
- Bias term is not regularized

Note

In scikit-learn, this model is implemented as `SGDClassifier`.

Example 4.2: Regularized Linear Classifier

- ▶ Train a regularized linear classifier with `SGDClassifier`
 - ▶ Use the MNIST dataset to distinguish the digit “5” from all others
 - ▶ Show how regularization improves generalization
 - ▶ Demonstrate prediction on a selected handwritten digit
- This example illustrates how a sparse linear decision boundary can work well for binary digit classification.

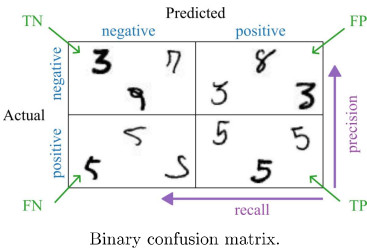
Performance Measures for Classification

- Evaluating a classifier is often more challenging than evaluating a regressor.
- Questions we might ask:
- ▶ How many predictions were correct?
 - ▶ What kinds of mistakes were made?
 - ▶ Which classes are most often confused?
- A common tool for answering these questions is the:

Confusion Matrix

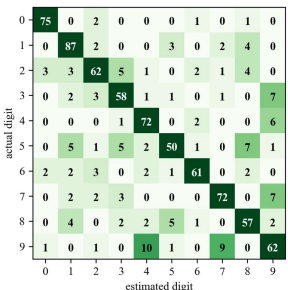
Binary Confusion Matrix

- Two common error types:
- ▶ Type I Error:
 - ▶ False Positive (FP)
 - ▶ Type II Error:
 - ▶ False Negative (FN)
- A confusion matrix summarizes:
- ▶ True Positives (TP)
 - ▶ True Negatives (TN)
 - ▶ False Positives (FP)
 - ▶ False Negatives (FN)



Confusion Matrix

- Rows correspond to:
- ▶ Actual classes
- Columns correspond to:
- ▶ Predicted classes
- A good classifier has:
- ▶ Large values on the diagonal
 - ▶ Small values elsewhere
- Off-diagonal entries indicate misclassifications.



Confusion Matrix Terminology

A confusion matrix summarizes classifier predictions.

- ▶ Rows = actual class
- ▶ Columns = predicted class

▶ TP: True Positive
▶ TN: True Negative
▶ FP: False Positive
▶ FN: False Negative

A perfect classifier contains values only along the main diagonal.

TN

FP

TN

FN

TP

FN

TN

FP

TN

Example 4.3: Binary Confusion Matrix

- ▶ Train a binary classifier to detect the digit “5”
- ▶ Generate a confusion matrix
- ▶ Compute:
 - ▶ True Positives (TP)
 - ▶ True Negatives (TN)
 - ▶ False Positives (FP)
 - ▶ False Negatives (FN)
- ▶ Examine examples of:
 - ▶ Missed 5s (false negatives)
 - ▶ Incorrectly detected 5s (false positives)
- ▶ Use the confusion matrix to understand where the classifier fails

Accuracy

Accuracy is defined as

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

or

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ▶ Simple and intuitive
- ▶ Often the first metric reported
- ▶ Does not distinguish between error types

Why Accuracy Isn't Enough

- A classifier may achieve high accuracy while still making costly mistakes.
- Examples:
- ▶ Credit card fraud detection
 - ▶ Medical diagnosis
 - ▶ Network intrusion detection
- A false negative may be much more costly than a false positive.
- We therefore introduce:

Precision Recall

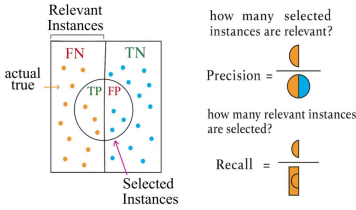
Selected and Relevant Instances

Definitions:

Selected Instances = $TP + FP$

Relevant Instances = $TP + FN$

These quantities form the basis of precision and recall.



Precision

Precision answers:
Of all positive predictions, how many were correct?

$$\text{Precision} = \frac{TP}{TP + FP}$$

- High precision implies:
- Few false positives
 - Positive predictions are trustworthy

Recall

Recall answers:
Of all actual positives, how many were found?

$$\text{Recall} = \frac{TP}{TP + FN}$$

- High recall implies:
- Few false negatives
 - Important cases are not missed

F1 Score

The F1 score balances precision and recall.

$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Equivalent form:

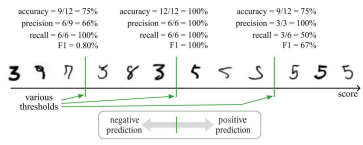
$$F_1 = \frac{TP}{TP + \frac{FP + FN}{2}}$$

Useful when both precision and recall are important.

Precision–Recall Tradeoff

Changing the classification threshold changes:

- Precision
 - Recall
 - F1 score
- Higher threshold:
- Higher precision
 - Lower recall

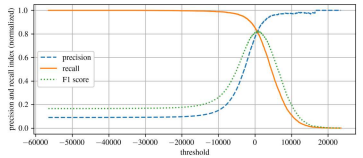


Choosing a Threshold

Threshold selection depends on the application.

- Examples:
- Medical diagnosis → high recall
 - Spam detection → high precision

The F1 score often helps identify a good compromise.

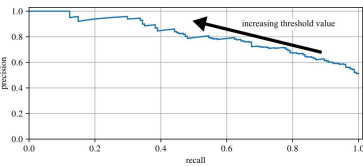


Precision–Recall Curve

The precision–recall curve summarizes classifier performance across all thresholds.

- Better classifiers typically have:
- Higher precision
 - Higher recall
 - Larger AUC-PR

Applications determine the desired operating point.



Example 4.4: Accuracy, Precision, and Recall

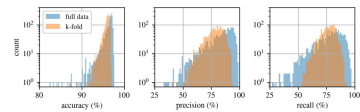
- Train a binary classifier to detect the digit “5”
- Compute:
 - Accuracy
 - Precision
 - Recall
 - F1 Score
- Compare manual and sklearn calculations
- Visualize threshold effects
- Explore precision–recall tradeoffs

k-fold Cross-Validation

- Training an SGD classifier does not always produce identical results.
- Reasons:
- Random initialization
 - Random ordering of samples
 - Stochastic optimization
- Possible solutions:
- More training epochs
 - More data
 - Cross-validation
- Goal:
- Obtain a more reliable estimate of model performance

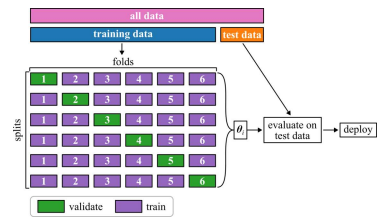
Performance Variability

- Repeated training can produce different results.
- Observations:
- Accuracy varies
 - Precision varies
 - Recall varies
- Cross-validation reduces this variability and produces more stable estimates.



How k-fold Cross-Validation Works

- Procedure:
1. Split data into k folds
 2. Train on $k - 1$ folds
 3. Validate on the remaining fold
 4. Repeat for all folds
 5. Average the results
- Advantages:
- Uses all available data
 - Better estimate of performance
 - Reduces dependence on a single train/test split



Example 4.5: k-fold Cross-Validation

- Train a binary classifier to detect the digit “5”
- Repeatedly train an SGD classifier
- Observe variability in:
 - Accuracy
 - Precision
 - Recall
- Apply:

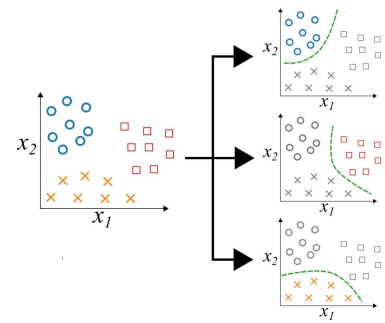
```
sk.model_selection.cross_val_predict
```
- Demonstrate how k -fold cross-validation provides more reliable performance estimates

Multiclass Classification

- Binary classifiers distinguish between:
- 2 classes
- Multiclass classifiers distinguish between:
- $N > 2$ classes
- For MNIST:
- 10 classes
- corresponding to the digits:
- $0, 1, 2, \dots, 9$
- A common strategy is to combine multiple binary classifiers.

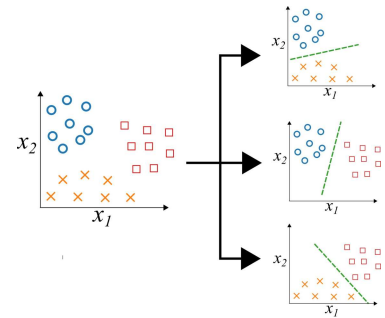
One-vs-Rest (OvR)

- One-vs-Rest trains:
- N
- binary classifiers.
- For MNIST:
- 0-detector
 - 1-detector
 - 2-detector
 - ...
 - 9-detector
- The class with the highest score is selected.



One-vs-One (OvO)

- One-vs-One trains a classifier for every pair of classes.
- Number of classifiers:
- $$\frac{N(N-1)}{2}$$
- For MNIST:
- $$\frac{10(10-1)}{2} = 45$$
- classifiers.
- The class with the most wins is selected.



Example 4.6: Multiclass Regularized Linear Classifier

- Extend `SGDClassifier` to multiclass classification
- Compare:
 - One-vs-Rest (OvR)
 - One-vs-One (OvO)
 - Scikit-Learn automatic strategy
- Train classifiers on the MNIST dataset
- Measure computational cost and performance
- Compare strategies for distinguishing the 10 digit classes

Many of the metrics used for binary classification extend naturally to multiclass problems.

Common evaluation tools include:

- Confusion matrices
- Accuracy
- Precision and recall
- Error analysis

For multiclass problems, the confusion matrix is often the most informative starting point.

- Train a One-vs-One classifier using SGD
- Perform 3-fold cross-validation
- Construct a multiclass confusion matrix
- Visualize:
 - Raw classification counts
 - Normalized classification errors
- Identify commonly confused digits
- Analyze classifier weaknesses
- Use the results to guide model improvements

Multiclass Confusion Matrix

Rows correspond to:

- Actual digit

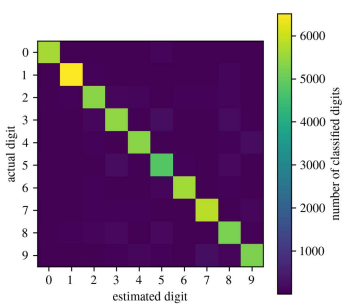
Columns correspond to:

- Predicted digit

A good classifier has:

- Large values on the diagonal
- Small values elsewhere

The MNIST classifier performs well because most entries lie on the diagonal.



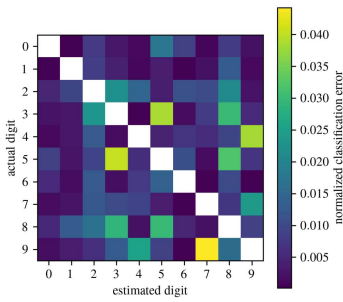
Normalized Error Matrix

To better visualize mistakes:

- Normalize each row
- Remove the diagonal entries
- Display only classification errors

This highlights which digits are most often confused.

Brighter regions correspond to larger error rates.



Analyzing Individual Errors

The classifier frequently confuses:

$3 \leftrightarrow 5$

Reasons:

- Similar handwritten shapes
- Small stroke differences
- Sensitivity to translation and rotation

Improvement strategies:

- Better preprocessing
- Additional features
- Nonlinear models

digits classified as a 3		digits classified as a 5	
correctly classified as 3s		correctly classified as 5s	
incorrectly classified as 3s		incorrectly classified as 5s	